Bilan hebdomadaire TER

Semaine du 15 Juin 2026

ABBES Samy & AREZKI Célia

1 Contexte général

Cette semaine a été consacrée à la stabilisation finale de notre plateforme d'expérimentation ByzFL, à l'organisation des résultats produits par les différentes campagnes de tests, ainsi qu'à la mise en place d'une chaîne de comparaison en deux niveaux : d'abord comparer tous les agrégateurs pour chaque attaque, puis comparer uniquement les meilleures courbes retenues pour chaque attaque afin d'identifier l'attaque la plus difficile et le meilleur agrégateur associé.

Nous avons travaillé en binôme de manière continue sur la base de code, sur les scénarios JSON, sur les résultats générés et sur les scripts de comparaison. Cette semaine a aussi été marquée par plusieurs opérations Git importantes : pull, rebase, correction de conflit sur un CSV de résultats et push final vers le dépôt distant.

2 Objectifs de la semaine

Les objectifs fixés au début de cette semaine étaient les suivants :

— ajouter de nouveaux scénarios et enrichir ceux déjà présents dans le dépôt ;

— organiser les scénarios par attaque et par distribution pour rendre l'arborescence plus lisible ;

— lancer les scénarios IID et Non-IID de façon séparée afin d'éviter les conflits de fichiers de résultats ;

— créer deux comparateurs complémentaires : un comparateur attaque/agrégateur et un comparateur final basé sur la meilleure courbe de chaque attaque ;

— sécuriser la synchronisation Git entre les résultats produits localement et les résultats du dépôt distant ;

— préparer une version exploitable dans le rapport hebdomadaire avec des emplacements pour les figures finales.

3 Historique des modifications et commits

Pour garder une trace claire du travail réalisé, nous avons repris l'historique Git de la semaine et identifié les changements les plus importants.

3.1 Commit de structuration des scénarios non-IID

Une première série de modifications a concerné la normalisation des configurations non-IID. Nous avons corrigé les scénarios pour homogénéiser les paramètres et garantir la cohérence des comparaisons, notamment en imposant une base commune de **5 clients honnêtes** et **2 attaquants** sur les scénarios concernés.

Cette étape a touché de nombreux fichiers JSON dans les dossiers des scénarios : SignFlipping, Inner Product Manipulation, Opt-IPM, ALIE, Opt-ALIE, Mimic, Infinity et Gaussian.

3.2 Commit lié au lancement des expériences Mimic

Un autre commit majeur de la semaine a ajouté et validé les scénarios Mimic en IID, avec tous les agrégateurs retenus. Cela a généré de nouveaux graphiques de sortie et a enrichi le fichier global de résultats 'resultats$_{c}ombines.csv$'.

3.3 Commit de modification du lanceur d'expériences

Nous avons ensuite modifié le lanceur principal pour améliorer l'organisation des fichiers de sortie. Le but était d'éviter les collisions entre expériences IID et Non-IID, en attribuant des noms plus précis aux CSV générés selon l'attaque et la distribution.

3.4 Commit des commandes Google Colab

Enfin, un commit a porté sur le fichier de commandes Colab. Nous avons clarifié la procédure d'exécution GPU et la séquence Git à suivre lorsque les expériences sont relancées depuis Colab.

4 Organisation du dépôt et des scénarios

L'un des apports les plus importants de cette semaine a été l'organisation définitive des scénarios par attaque.

Chaque famille d'attaque est désormais rangée dans un sous-dossier dédié, avec deux sous-ensembles :

— `iid/` pour les scénarios à distribution identique entre les clients ;

— `non_iid/` pour les scénarios hétérogènes.

Cette nouvelle structure concerne notamment :

— Sign Flipping,

— Inner Product Manipulation (IPM),

— Optimal Inner Product Manipulation (Opt-IPM),

— A Little Is Enough (ALIE),

— Optimal A Little Is Enough (Opt-ALIE),

— Mimic,

— Infinity,

— Gaussian.

Nous avons également harmonisé les noms des agrégateurs utilisés dans les scénarios : Average, Median, TrMean, Geometric Median, Krum, Multi-Krum, Centered Clipping, MDA, MoNNA, Meamed, CAF et SMEA.

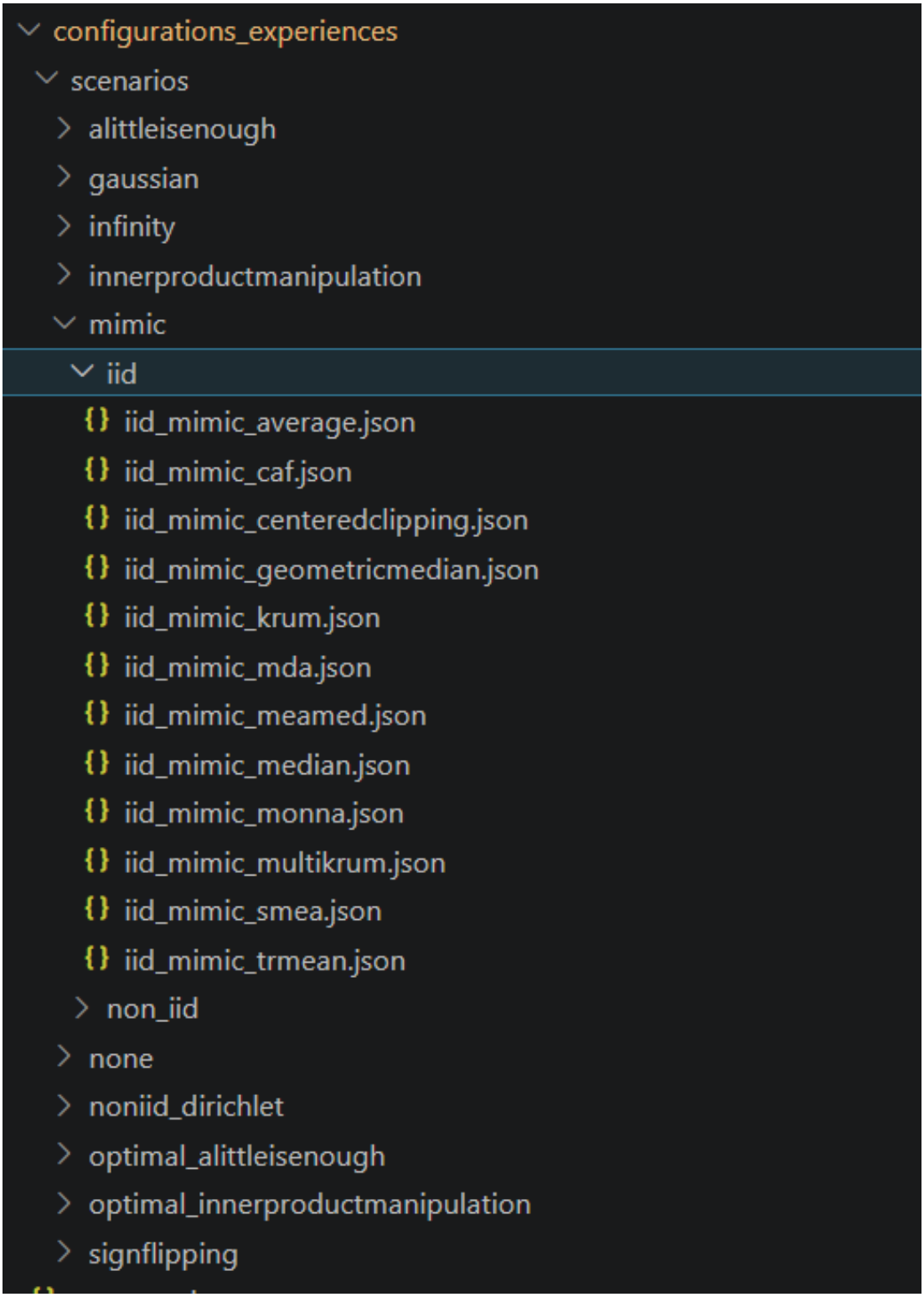


FIGURE 1 – Organisation finale des scénarios par attaque avec sous-dossiers IID et Non-IID

5 Passage à deux niveaux de comparaison

Cette semaine a surtout servi à mettre en place une logique d'analyse plus claire des résultats.

5.1 Premier niveau : tous les agrégateurs pour chaque attaque

Le premier script de comparaison, `comparer_attaques_agregateurs.py`, trace pour chaque attaque les courbes de tous les agrégateurs disponibles. Cela permet de visualiser la robustesse de chaque défense face à une attaque donnée, en gardant un axe des ordonnées normalisé entre 0 et 1 et des couleurs cohérentes d'un graphique à l'autre.

Le script produit également un résumé CSV des meilleurs agrégateurs par attaque, avec un classement interne selon l'accuracy finale.

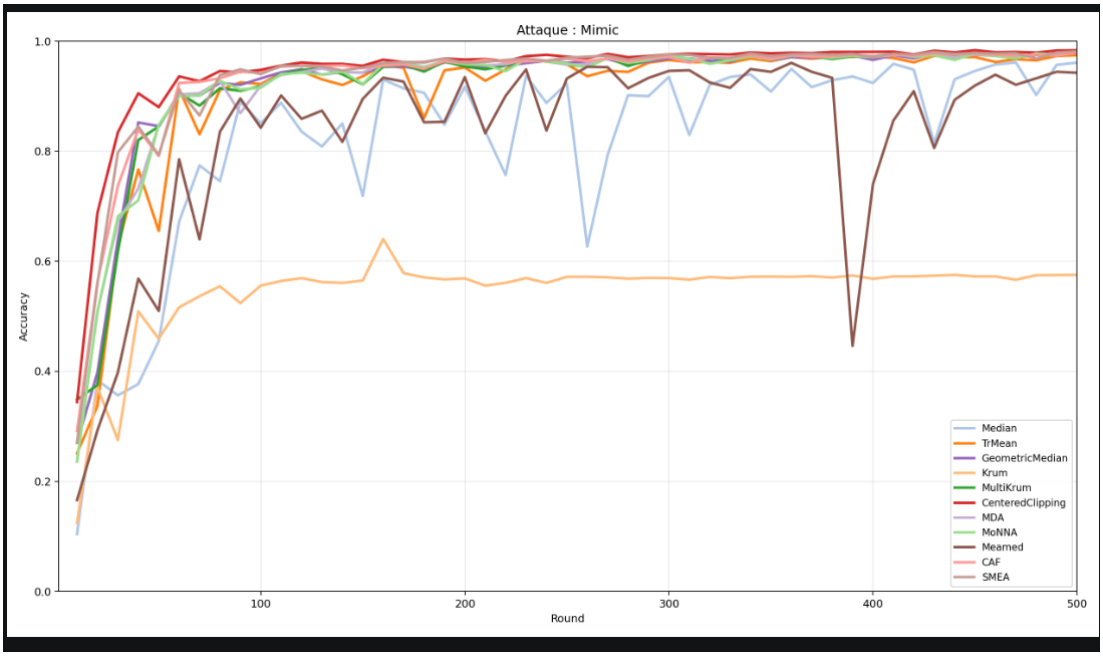


FIGURE 2 – Courbes de tous les agrégateurs pour une attaque donnée

5.2 Deuxième niveau : meilleure courbe retenue pour chaque attaque

Le second script, `comparer_meilleures_attaques.py`, prend le meilleur agrégateur pour chaque attaque, extrait sa courbe complète et compare ensuite ces meilleures courbes sur un seul graphique final.

L'idée est de répondre à deux questions distinctes :

— quelle est la pire attaque parmi celles testées ?

— quel est le meilleur agrégateur pour chaque attaque, puis globalement ?

Ce deuxième niveau fournit donc un classement final plus synthétique et plus exploitable pour le rapport.

courbe_comparaison_meilleures_attaques.png

FIGURE 3 – Comparaison finale des meilleures courbes retenues pour chaque attaque